

MiniPrologProto Manual and IDE Tutorial

Project: MiniPrologProto (Java) + ColumnDB + HTN planner + PyQt IDE

Build artifact: precompiled `out/` included in the distributed zip

Date: 2026-01-03

Table of contents

1. Quick start
2. What this Prolog supports (syntax and semantics)
3. Built-ins reference (core, JSON, POJOs, dynamic predicates)
4. REPL guide
5. IDE guide (PyQt6)
6. Prototype OO layer
7. ColumnDB (tables, indexes, select)
8. JSON-driven joins (``join_json/2``)
9. HTN planner (demo domain and profiles)
10. Step-by-step tutorial
11. Troubleshooting and tips

1) Quick start

Prerequisites

- **Java:** Java 11+ recommended (uses `var` in some entrypoints).
- **Python:** Python 3.9+ recommended for the IDE.
- **IDE dependency:** `PyQt6` (`pip install PyQt6`).

Run the REPL (recommended first)

From the repo root:

```
./repl.sh          # full stack (OO + ColumnDB + Planner)
./repl.sh --sample # minimal sample program (append/3 + core engine)
```

Run self-tests

```
./selftest.sh
```

Start the GUI IDE

```
python3 prolog_ide.py
```

2) What this Prolog supports

This is a deliberately small Prolog-style engine (unification + backtracking) with a pragmatic text syntax.

Statements

- **Query:** `?- Goal.`
- **Directive:** `:- Goal.` (executed immediately; useful in files)
- **Fact:** `head.`
- **Rule:** `head :- body.`

Terms

- **Atoms:** `foo`, `bar\_baz`, `null`, `true`

- **Numbers:** ``123``
- **Floats:** produced by JSON as ``double(Atom("2.5"))`` (see JSON section)
- **Variables:** start with uppercase or ``_`` (e.g. ``X``, ``Person``, ``_Tmp``)
- **Compound terms:** ``parent(alice, bob)``

Lists

- Empty list: ``[]``
- Proper list: ``[a,b,c]``
- Cons form (internal): ``.(Head, Tail)``
- Improper list: ``[H|T]``

Operators in goals

- Conjunction: ``A, B``
- Disjunction: ``A ; B``
- If-then: ``A -> B``
- Negation-as-failure: ``\+ Goal``
- Cut: ``!``

Comments

- ``%`` comment to end of line
- ``//`` comment to end of line

Strings and quoting

- Both ``single quoted`` and ``"double quoted"`` literals are read as **atoms** by the REPL parser.
- When you need a **JSON string term** (the Prolog representation ``str(AtomPayload)``), write:

```
`str(`{"k":1}`)
```

The inner quotes are JSON; the outer quotes are Prolog atom quoting.

3) Built-ins reference

Some operations are implemented as special forms in the engine (e.g. conjunction/disjunction), and others are dispatched as named built-ins.

3.1 Control / meta

- ``call(Goal)``
- ``once(Goal)`` - succeeds at most once (commits to the first solution)

3.2 Arithmetic and comparisons

- ``X is Expr``
- Numeric comparisons: ``:=``, ``\=``, ``<``, ``=<``, ``>``, ``>=``

Arithmetic expressions support: ``+`` ``-`` ``*`` ``//`` ``mod`` and unary ``-``.

3.3 Aggregation

- ``findall(Template, Goal, List)``
- ``bagof(Template, Goal, List)``
- ``setof(Template, Goal, List)``

3.4 Dynamic database

- ``assertz(Clause)`` - add a fact/rule at the end of the predicate
- ``retract(Clause)`` - remove (nondeterministically) matching clauses

In the REPL, entering a fact/rule like ``p(a).`` is equivalent to calling ``assertz/1`` internally.

3.5 JSON <-> Term

- ``json_parse(JSONText, Term)``
- ``json_stringify(Term, JSONText)``

``JSONText`` can be:

- an atom: ``Atom("...")`` as typed in the REPL, e.g. ``{"a":1}``, or
- a JSON string term: ``str("{\"a\":1})``

Term shapes produced by ``json_parse/2``:

- Object: ``obj(List)`` where ``List`` is a Prolog list of ``kv(Key,Val)``
- Array: ``arr(List)``
- String: ``str(AtomPayload)``

- Booleans/null: atoms ``true``, ``false``, ``null``
- Integer: ``NUM``
- Float: ``double(Atom("2.5"))``

3.6 JSON <-> Java POJO (reflection)

- ``from_json(JSONText, ClassName, Ref)``
- ``to_json(Ref, JSONText)``

Where:

- ``ClassName`` is a string/atom, e.g. ``str('com.mycompany.prolog.Person')``
- ``Ref`` is a heap reference term: ``ref(N)``

Safety constraints:

- Only classes under allowed prefixes are instantiable (default includes ``com.mycompany.prolog.``).
- Uses a public no-arg constructor and public non-static, non-final fields.

3.7 Heap hygiene (important with POJOs)

- ``release_ref(Ref)``
- ``clear_heap``
- ``heap_size(N)``

Use ``release_ref/1`` when you no longer need a POJO reference.

3.8 AIML helpers (optional)

- ``aiml_tokenize/2``
- ``aiml_detokenize/2``

(Used by the included AIML demo/studio Java entrypoints.)

4) REPL guide

Start it:

```
./repl.sh          # full stack
./repl.sh --sample # minimal stack
```

Prompts

- Main prompt: `?-`
- Continuation prompt (multi-line statement): `|`

Multiple solutions

When there are multiple answers, the REPL prints one and then waits:

- type `;` and press Enter to see the next solution
- press Enter (or type anything else) to stop

REPL commands

- `:help`
- `:quit` / `:q`
- `:reset` (rebuilds the program; keeps the chosen stack)
- `:stack sample|full`
- `:consult path/to/file.pl`
- `:predicates` (list predicate indicators currently loaded)

Consulting files

`:consult file.pl` reads and processes statements:

- facts/rules are `assertz`'d
- directives `:- Goal.` are executed at consult time
- queries `?- ...` inside files are ignored

5) IDE guide (PyQt6)

Start:

```
python3 prolog_ide.py
```

Layout

- **Left:** Examples and templates tree (double-click to load into the editor)
- **Top-right:** Editor tabs (Prolog-like text)
- **Bottom-right:** Console output + “Generated Java code” view
- **REPL controls:** Start/Stop REPL buttons + a one-line input box

Toolbar shortcuts

- New: Ctrl+N
- Open: Ctrl+O
- Save: Ctrl+S
- Generate Java: Ctrl+G
- Build: Ctrl+B
- Run Demo: Ctrl+R
- Start REPL: Ctrl+E
- Stop REPL: Ctrl+Shift+E

Typical IDE workflow

1. Double-click an example (e.g. **Database** → **JSON Join**) to load it.
2. Click **Start REPL**.
3. Paste lines into the REPL input (one line per send).
4. Iterate in the editor, save to a `.pl`, and use `:consult` in the REPL.

Note: The IDE is a lightweight front-end; it does not (yet) parse a whole buffer and stream it to the REPL automatically. Use the REPL input box for execution.

6) Prototype OO layer (OopPrologLib)

The full stack includes a small prototype-based OO substrate inside Prolog, inspired by JavaScript prototypes.

Initialization

```
?- oop_init.
```

Objects, slots, and prototypes

- Create an object from a prototype: ``clone(Proto, Obj).``
- Set a slot: ``put(Obj, Key, Value).``
- Get a slot (walks prototype chain): ``get(Obj, Key, Value).``
- Send a message (method lookup along prototype chain): ``send(Obj, Msg).``

Defining methods

```
?- defmethod(objectProto, name_of(Name), Self,
             get(Self, name, Name)).
```

Calling it:

```
?- clone(objectProto, 0),
   put(0, name, bob),
   send(0, name_of(N)).
```

7) ColumnDB (ColumnDbLib)

ColumnDB is built on the OO substrate (tables and the DB are OO objects).

Initialize ColumnDB

```
?- oop_init, columndb_init.
?- clone(dbProto, DB).
```

Create tables

Schema is a list of ``col(Name, Type, Opts)`` terms. Types are not heavily enforced (common: ``num``, ``int``, ``atom``).

```
?- send(DB, create_table(users,
                        [col(id,int,[]), col(name,atom,[])],
                        UsersT)).
```

Insert rows

Rows are ``row([col=value, ...])``:

```
?- send(UsersT, insert(row([id=1, name=ada]))).
?- send(UsersT, insert(row([id=2, name=bob]))).
```

Select rows

``send(Table, select(Proj, Filter, Opts, Rows)).``

- ``Proj``: ``all`` or a list of column names (e.g. ``[id,name]``)
- ``Filter``:
 - ``all``
 - ``where(Col, eq, Val)``
 - ``where(Col, ge, Val)``
 - ``and(F1,F2)``, ``or(F1,F2)``, ``not(F)``
- ``Opts``: ``opts(limit(all))`` or ``opts(limit(N))``

Example:

```
?- send(UsersT, select(all, where(id, ge, 2), opts(limit(all)), Rows)).
```

Count

```
?- send(UsersT, count(all, N)).
?- send(UsersT, count(where(id, eq, 1), N)).
```

Indexes

Hash indexes accelerate equality (``eq``) filters and joins.

```
?- send(UsersT, create_index(id, hash)).
```

8) JSON-driven joins (``join_json/2``)

ColumnDB supports a DB-side **single-key equi-join** driven by a JSON specification:

```
?- send(DB, join_json(JsonSpec, Rows)).
```

Where ``JsonSpec`` can be:

- a JSON string term: ``str("{...}")``, or
- an already-parsed JSON term (from ``json_parse/2``)

Minimal join spec (inner join)

Required fields:

- ``left`, `right``: table names
- ``on.left`, `on.right``: join key columns

Optional:

- ``type``: ``"inner" | "left" | "right" | "full"`` (default ``"inner"``)
- ``select``: projection per side
- ``where``: filter per side
- ``opts``: ``qualify`, `aliases`, `limit``

Example (left join with filtering and qualified output keys):

```
?- Json = str('{
  "type": "left",
  "left": "users",
  "right": "orders",
  "on": {"left": "id", "right": "user_id"},
  "select": {"left": ["id", "name"], "right": ["id", "total"]},
  "where": {"right": {"col": "total", "op": "ge", "val": 50}},
  "opts": {"qualify": true, "aliases": {"left": "u", "right": "o"}, "limit": 50}
}'),
send(DB, join_json(Json, Rows)).
```

Output shape: a list of ``row(Pairs)`` where keys are either qualified (``q(Alias,Col)``) or unqualified depending on ``opts.qualify``.

Limitations:

- only **single-key equality** joins
- no group-by / aggregation / sorting
- filters support ``eq`` and ``ge`` only in the JSON DSL

See ``README_JSON_JOINS.md`` for the full JSON schema.

9) HTN planner

The full stack includes:

- ``PlannerStrategiesLib`` (planner engine + strategies: caching, bounded DFS, auto-indexing)
- ``HtnDemoLib`` (a small demo domain + seed data + driver predicate)

One-line demo

```
?- demo_plan(indexed, Plan).
```

Profiles:

- ``small`` : plain runner + DFS
- ``cached`` : caches query results + bounded DFS
- ``indexed`` : auto-creates hash indexes for ``eq`/`in`` filters + bounded DFS

How the planner is structured

The demo domain defines:

- ``primitive(Task)``
- ``method(Ctx, Task, Subtasks)``
- ``op(Ctx, PrimitiveTask)`` (preconditions / interpreter)
- ``design_query(Need, q(Table,Proj,Filter,Opts))`` used by strategies to query the DB

If you want to build your own domain, start by copying ``HtnDemoLib`` patterns.

10) Step-by-step tutorial

You can do this entire tutorial in the REPL. (In the IDE, start the REPL and paste the same lines.)

10.1 Warm-up: unification

```
?- X = 42.
```

10.2 Define a tiny knowledge base

```
parent(ada, bob).  
parent(bob, cara).  
ancestor(X,Y) :- parent(X,Y).  
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Query:

```
?- ancestor(ada, Who).
```

10.3 Lists and recursion (append/3 exists in the sample program)

```
?- append([1,2], [3,4], X).
```

10.4 Arithmetic

```
?- X is 5 + 3 * 2.  
?- 7 =:= 3 + 4.  
?- 3 =\= 4.
```

10.5 JSON parsing and stringifying

```
?- json_parse(str('{"a":1,"b":[2,3],"c":"hi"}'), T).  
?- json_stringify(T, Back).
```

10.6 Map JSON to a Java POJO

```
?- from_json(str('{"name":"Ada","age":37}'),  
             str('com.mycompany.prolog.Person'),  
             Ref).  
?- to_json(Ref, JsonOut).  
?- release_ref(Ref).
```

10.7 ColumnDB: tables, inserts, selects

```
?- oop_init, columndb_init.  
?- clone(dbProto, DB),  
  send(DB, create_table(users, [col(id,int,[]), col(name,atom,[])], UsersT)).  
  
?- send(UsersT, insert(row([id=1, name=ada])),  
  send(UsersT, insert(row([id=2, name=bob])))).  
  
?- send(UsersT, select(all, where(id, ge, 2), opts(limit(all)), Rows)).
```

10.8 Join

```
?- send(DB, create_table(orders, [col(id,int,[]), col(user_id,int,[]), col(total,int,[])], OrdersT).  
  
?- send(OrdersT, insert(row([id=10, user_id=1, total=120])),  
  send(OrdersT, insert(row([id=11, user_id=1, total=40 ]))),  
  send(OrdersT, insert(row([id=12, user_id=3, total=9 ]))).  
  
?- send(OrdersT, create_index(user_id, hash)).  
  
?- Json = str('{"type":"left","left":"users","right":"orders","on":{"left":"id","right":"user_id"},  
  "select":{"left":["id","name"],"right":["id","total"]},  
  "where":{"right":{"col":"total","op":"ge","val":50}},  
  "opts":{"qualify":true,"aliases":{"left":"u","right":"o"},"limit":50}'),  
  send(DB, join_json(Json, Rows)).
```

10.9 HTN demo plan

```
?- demo_plan(indexed, Plan).
```

11) Troubleshooting and tips

REPL won't start

- Ensure Java is installed and on PATH: ``java -version``
- Ensure ``out/`` exists; if not: ``./build.sh``
- Use ``./repl.sh --sample`` to confirm the minimal stack runs.

Built-ins throw errors but you want the engine to continue

The engine can print or suppress builtin errors and stack traces via ``-D`` flags:

```
java -Dprolog.printBuiltinErrors=true -Dprolog.printBuiltinStacktraces=false -Dprolog
```

“dangling ref” / heap growth

If you store POJOs via ``from_json/3``, remember to call ``release_ref/1`` when done, or ``clear_heap``.

JSON parse errors

- Validate the JSON (commas, quotes, braces).
- In the REPL, wrap JSON in ``str(...)`` so it remains a single Prolog term.

Joins are slow

- Index the **right** join key: ``send(RightTable, create_index(join_key, hash)).``
- Filter early with ``where.left`` / ``where.right`` in the join JSON.

Appendix: Cheat sheet

REPL commands

- ``:help`` ``:consult file.pl`` ``:reset`` ``:stack sample|full`` ``:predicates`` ``:quit``

ColumnDB key calls

```
?- oop_init, columndb_init.  
?- clone(dbProto, DB).  
?- send(DB, create_table(Name, Schema, Table)).  
?- send(Table, insert(row([...]))).
```

```
?- send(Table, select(Proj, Filter, opts(limit(N)), Rows)).  
?- send(Table, count(Filter, N)).  
?- send(Table, create_index(Col, hash)).  
?- send(DB, join_json(JsonSpec, Rows)).
```